Lab.3

Zadanie 1

Program rejestruje handler dla sygnału SIGINT (Ctrl+C) za pomocą sigaction(). Zamiast kończyć działanie, program wypisuje komunikat „Odebrano SIGINT” i działa dalej.

```
  GNU nano 7.2
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
write(1, "Odebrano SIGINT\n", 16);
}

int main()
{
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGINT, &sa, NULL);

while (1)
{
printf("Pracuje... (Ctrl + C aby wyslac SIGINT)\n");
sleep(2);
    }
return 0;
}
```

```
root@debian:~# gcc zad1L3.c -o zad1L3
root@debian:~# ./zad1L3
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
^COdebrano SIGINT
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
Pracuje... (Ctrl + C aby wyslac SIGINT)
^\Quit
root@debian:~# _
```

Zadanie 2

Program wysyła sygnał SIGUSR1 do samego siebie funkcją raise() (lub kill(getpid(), SIGUSR1)). Zarejestrowany handler wypisuje komunikat po otrzymaniu sygnału.

```
  GNU nano 7.2
#include <stdio.h>
#include <signal.h>

void handler(int sig)
{
write(1, "Odebrano SIGUSR1\n", 17);
}

int main()
{
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGUSR1, &sa, NULL);

printf("Wysylam SIGUSR1, do samego siebie\n");
raise(SIGUSR1);
printf("Koniec\n");
return 0;
}
```

```
root@debian:~# gcc zad2L3.c -o zad2L3
zad2L3.c: In function ‘handler’:
zad2L3.c:6:1: warning: implicit declaration of function ‘write’; did you mean ‘fwrite’? [
    6 | write(1, "Odebrano SIGUSR1\n", 17);
      | ^~~~~
      | fwrite
root@debian:~# ./zad2L3
Wysylam SIGUSR1, do samego siebie
Odebrano SIGUSR1
Koniec
root@debian:~#
```

Zadanie 3

Proces wysyła serię sygnałów SIGUSR1 i SIGUSR2 do drugiego procesu. Odbiorca ma osobne handlery dla obu sygnałów i zlicza, ile sygnałów każdego typu odebrał.

```
  GNU nano 7.2
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

int licznik1 = 0;
int licznik2 = 0;

void handler(int sig) {
if (sig == SIGUSR1) licznik1++;
else if (sig == SIGUSR2) licznik2++;
}

int main() {
pid_t pid = fork();

if (pid == 0) {
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGUSR1, &sa, NULL);
sigaction(SIGUSR2, &sa, NULL);

sleep(3);
printf("Odbiorca: SIGUSR1 = %d, SIGUSR2 = %d\n", licznik1, licznik2);
} else {
sleep(1);
for (int i = 0; i < 3; i++) {
kill(pid, SIGUSR2);
usleep(1000000);
 }
wait(NULL);
}
return 0;
}
```

```
root@debian:~# gcc -std=gnu99 zad3L3.c -o zad3L3
root@debian:~# ./zad3L3
Odbiorca: SIGUSR1 = 0, SIGUSR2 = 1
root@debian:~#
```

Zadanie 4

Program próbuje zarejestrować handler dla SIGKILL. **Zadanie podchwytliwe:** sygnału SIGKILL (9) nie da się przechwycić, zablokować ani zignorować sigaction() zwraca błąd (-1, EINVAL), a proces zawsze kończy się natychmiast. To gwarancja systemu, aby administrator mógł zakończyć każdy proces.

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
write(1, "Handler SIGKILL\n", 16);
}

int main()
{
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

if (sigaction(SIGKILL, &sa, NULL) == -1)
{
perror("sigaction SIGKILL");
printf("NIe da si przechwycic SIGKILL!\n");
} else {
printf("Handler zarejestowany (nie powinno sie zdarzyc)\n");
}
return 0;
}
```

```
root@debian:~# gcc zad4L3.c -o zad4L3
root@debian:~# ./zad4L3
sigaction SIGKILL: Invalid argument
NIe da si przechwycic SIGKILL!
root@debian:~# _
```

Zadanie 5

Program demonstruje różnicę między sygnałami maskowalnymi a niemaskowalnymi. Sygnał maskowalny (np. SIGINT) można zablokować przez sigprocmask() czeka wtedy jako pending i zostaje obsłużony po odblokowaniu. **Sygnałów niemaskowalnych (SIGKILL, SIGSTOP) nie da się zablokować** system ignoruje próbę ich maskowania i działają natychmiast.

```c
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
sigset_t set;
sigemptyset(&set);
sigaddset(&set, SIGINT);
sigaddset(&set, SIGKILL);

sigprocmask(SIG_BLOCK, &set, NULL);
printf("Probuje zablokowac SIGINT i SIGKILL\n");

sigset_t pending;
printf("Nacisni Ctrl+C - SIGINIT zostanie wstrzymany...\n");
sleep(5);

sigpending(&pending);
if (sigismember(&pending, SIGINT))
printf("SIGINIT jest pending (zamaskowany skutecznie)\n");
if (sigismember(&pending, SIGKILL))
printf("SIGKILL pending\n");
else
printf("SIGKILL NIE zostal zamaskowany (niemaskowalny!)\n");

printf("Odblokowuje SIGINIT...\n");
sigprocmask(SIG_UNBLOCK, &set, NULL);
printf("Koniec\n");
return 0;
}
```

```
root@debian:~# gcc zad5L3.c -o zad5L3
root@debian:~# ./zad5L3
Probuje zablokowac SIGINT i SIGKILL
Nacisni Ctrl+C - SIGINIT zostanie wstrzymany...
^CSIGINIT jest pending (zamaskowany skutecznie)
SIGKILL NIE zostal zamaskowany (niemaskowalny!)
Odblokowuje SIGINIT...

root@debian:~# _
```

Zadanie 6

Program tworzy dwa procesy: producent i konsument. Producent po przygotowaniu danych wysyła sygnał SIGUSR1 do konsumenta funkcją kill(). Konsument w handlerze odbiera sygnał i wypisuje komunikat „Dane gotowe”.

```
 GNU nano 7.2
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <sys/wait.h>

void handler(int sig) {
write(1, "Konsument: Dane gotowe\n", 23);
}

int main() {
pid_t pid = fork();

if (pid == 0) {
struct sigaction sa;
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;
sigaction(SIGUSR1, &sa, NULL);

printf("Konsument:czekam na dane...\n");
pause();
} else {
printf("Producent: przygotowuje dane...\n");
sleep(2);
printf("Producent: dane gotowe, wysylam sygnal\n");
kill(pid, SIGUSR1);
wait(NULL);
}
return 0;
}
```

```
root@debian:~# gcc zad6L3.c -o zad6L3
root@debian:~# ./zad6L3
Producent: przygotowuje dane...
Konsument:czekam na dane...
Producent: dane gotowe, wysylam sygnal
Konsument: Dane gotowe
root@debian:~# _
```